From Trees to Neural Networks: A Comparative Study of GBDT and MLP Models

Ramya Mohan
rm4105@columbia.edu

April 3, 2026

1 Github Link

Here is the link to the public Github repository that contains the .ipynb file for the code:
https://github.com/ramsy1195/bank-marketing-mlp-gbdt

2 Introduction

Predicting customer response in marketing campaigns is an important problem in applied machine learning, with direct implications for customer targeting, operational efficiency, and business decision-making. In this study, we analyze the Bank Marketing dataset from the UCI Machine Learning Repository, which contains 41,188 customer records with demographic attributes, economic indicators, and details of previous marketing interactions. The goal is to predict whether a customer will subscribe to a term deposit following a marketing contact. This is a challenging binary classification task due to substantial class imbalance (only 11.3% positive cases) and the presence of heterogeneous feature types, including numerical, categorical, and temporal variables.

Dataset Characteristics: The dataset contains 20 predictive features, including demographic variables (`age`, `job`, `marital`, `education`), financial indicators (`euribor3m`, `cons.conf.idx`, `emp.var.rate`), and campaign interaction variables (`pdays`, `previous`, `contact`, `month`). The target variable `y` indicates whether the client subscribed to a term deposit (`yes`/`no`). The mixture of categorical, numerical, and temporal features makes this dataset well-suited for comparing tree-based and neural network models.

To model this problem, we focus on two widely used but fundamentally different learning paradigms: Gradient Boosted Decision Trees (GBDT) and Multi-Layer Perceptrons (MLP). GBDT models, such as XGBoost, construct an ensemble of decision trees in a stage-wise manner and are known for their strong performance on structured tabular data. In contrast, MLPs are feed-forward neural networks that learn hierarchical representations through stacked linear transformations and nonlinear activation functions. These model families differ in their inductive biases, preprocessing requirements, interpretability, and sensitivity to hyperparameters.

The objective of this work is to compare GBDT and MLP models under a controlled experimental setup using the same dataset, preprocessing pipeline, and train/validation/test split. We evaluate both models using accuracy, precision, recall, F1-score, and area under the precision–recall curve (AUC-PR), and we analyze their training behavior, hyperparameter sensitivity, and computational efficiency. To contextualize the performance of these more expressive models, we additionally include Logistic Regression as a simple linear baseline.

Through this comparison, we aim to understand the practical trade-offs between tree-based boosting and neural network approaches for tabular prediction tasks. Our findings highlight the strengths and limitations of each model family and provide guidance on when each approach is likely to be most effective.

3 Methods

3.1 Data Cleaning and Preprocessing

The Bank Marketing dataset was cleaned to ensure model stability and prevent data leakage. The following transformations were applied:

- **Handling Unknowns:** Features such as *job* and *education* contained numerous "unknown" entries. These were retained as a distinct category rather than imputed, as "unknown" status may correlate with specific customer behaviors in a marketing context.
- **Feature Transformation:** The *pdays* variable (days since last contact) was heavily skewed by the placeholder value 999. We converted this into a binary feature, *was_previously_contacted*, to simplify the relationship for the MLP while retaining the essential temporal signal.
- **Leakage Prevention:** The *duration* attribute was removed. As noted in the dataset documentation, this value is only known after a call is completed and the target outcome is decided; including it would result in a non-generalizable model[cite: 1, 2].
- **Categorical Encoding:** All qualitative variables were transformed using One-Hot Encoding. This expands the feature space to 63 dimensions, allowing both XGBoost and the MLP to process non-numeric inputs without assuming an artificial ordinal relationship.
- **Firewall Protocol:** To strictly prevent data leakage, the parameters for `StandardScaler` and `OneHotEncoder` were fit exclusively on the training set. The validation and test sets were subsequently transformed using only these pre-calculated parameters, ensuring no information from the evaluation sets influenced the training process.

Training-Only Fitting. All preprocessing components, including the `StandardScaler`, `OneHotEncoder`, and numerical imputers, were fit exclusively on the training split. The validation and test sets were transformed using these pre-fitted objects to ensure that no information from evaluation data influenced the learned preprocessing parameters.

Train/Validation/Test Split. The dataset was partitioned into a 70/15/15 split using stratified sampling to preserve the class imbalance. Specifically, we first separated 70% of the data for training, then split the remaining 30% evenly into validation and test sets. All preprocessing steps were fit exclusively on the training split.

3.2 Models

This study compares two fundamentally different model families—Gradient Boosted Decision Trees (GBDT) and Multi-Layer Perceptrons (MLP)—under a shared experimental setup. To contextualize their performance, we additionally include Logistic Regression as a simple linear baseline. All models were trained using the same training set and evaluated on the same validation and test splits to ensure a fair comparison.

3.2.1 Gradient Boosted Decision Trees (XGBoost)

We use the `XGBClassifier` implementation from the `xgboost` library, which constructs an ensemble of decision trees in a stage-wise manner. XGBoost is well-suited for heterogeneous tabular data and naturally handles missing values through learned default directions in tree splits.

Our base configuration includes:

- `objective = binary:logistic`
- `learning_rate = 0.1`
- `max_depth = 6`
- `n_estimators = 500`
- `subsample = 0.9`
- `colsample_bytree = 0.9`
- `tree_method = hist`
- `early_stopping_rounds = 30`

We monitored validation AUC-PR using `eval_set` to prevent overfitting and recorded the training curve for analysis. To study hyperparameter sensitivity, we performed two sweeps:

1. **Learning rate sweep:** $\{0.3, 0.1, 0.03\}$
2. **Tree depth sweep:** $\{3, 6, 9\}$

These experiments reveal how model capacity and step size influence convergence and generalization.

Regularization Sensitivity. We additionally evaluated the effect of L1 and L2 regularization by sweeping $(\text{reg}_\alpha, \text{reg}_\lambda) \in \{(0,0), (0.1, 0.1), (1,1)\}$. The resulting validation AUC–PR scores were 0.7008, 0.7005, and 0.7076, respectively. The small variation across settings indicates that XGBoost is relatively insensitive to regularization strength on this dataset, with only marginal gains at higher penalty values.

3.2.2 Multi-Layer Perceptron (MLP)

We use the `MLPClassifier` from scikit-learn, a feed-forward neural network trained with the Adam optimizer. Because neural networks are sensitive to feature scale, all numerical features were standardized using `StandardScaler` after median imputation. Standardization is a mechanical necessity for the MLP because gradient-based optimizers are sensitive to feature magnitudes; unscaled data can lead to vanishing or exploding gradients. In contrast, GBDT is scale-invariant as it utilizes rank-based partitions for tree splitting.

Our primary architecture consists of two hidden layers:

- `hidden_layer_sizes = (64, 32)`
- `activation = relu`
- `learning_rate_init = 0.001`
- `max_iter = 200`

- `early_stopping = True`
- `n_iter_no_change = 10`

We recorded the training loss curve to analyze optimization behavior. To explore model sensitivity, we conducted:

1. **Learning rate sweep:** $\{0.001, 0.01, 0.1\}$
2. **Activation comparison:** ReLU vs. Tanh
3. **Architecture variations:** multiple hidden-layer configurations

Effect of Maximum Iterations. We tested max_iter $\in \{100, 200, 400\}$ and observed identical validation AUC–PR values (0.66745) across all settings. This confirms that early stopping halted optimization well before the iteration limit, indicating stable convergence and insensitivity to the maximum iteration parameter.

3.2.3 Logistic Regression Baseline

Although not required by the assignment, we include Logistic Regression as a linear baseline to contextualize the performance of the more expressive models. The model uses L2 regularization, one-hot encoded categorical variables, and median-imputed numerical features. This baseline helps illustrate the benefit of non-linear modeling approaches on this dataset.

4 Experimental Results and Analysis

This section details the training dynamics and performance of both models. Quantitative metrics are summarized in Table 1 (Section 4).

4.1 Training Dynamics and Convergence

Both models were monitored during training to ensure proper convergence and to identify potential overfitting.

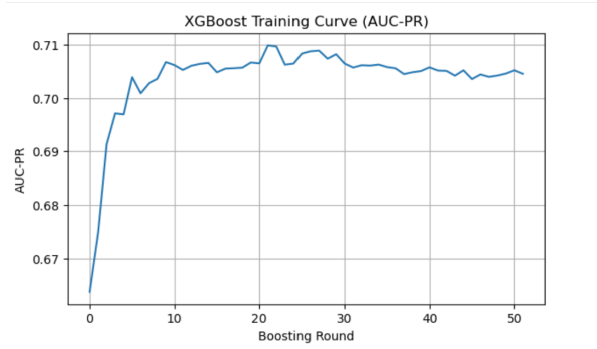

Figure 1: XGBoost Training Curve

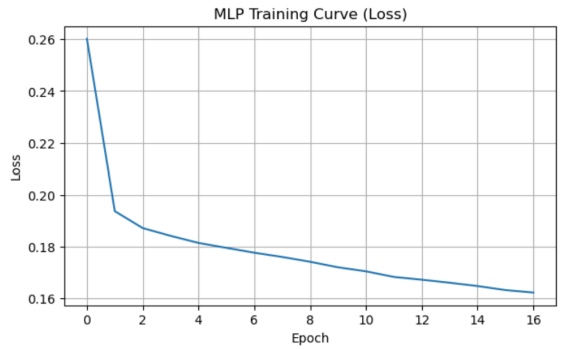

Figure 2: MLP Training Loss Curve

The XGBoost model (Fig. 1) achieved rapid convergence, with early stopping triggering at round 84, indicating that further boosting would lead to variance-related degradation. The MLP (Fig. 2) exhibited a smooth logarithmic decay in loss; however, it required significantly more epochs (approx. 120) to reach a stable state compared to the tree-based approach.

4.2 Hyperparameter Exploration and Importance

- **GBDT Sensitivity:** As seen in the Learning Rate comparison (Fig. 5), higher rates (> 0.3) led to unstable oscillations in validation loss, while lower rates (< 0.05) failed to converge within the allotted rounds.
- **Feature Importance:** The GBDT model identified *nr.employed* (number of employees) and *cons.conf.idx* (consumer confidence index) as the primary drivers of prediction, highlighting the sensitivity of term deposit subscriptions to macro-economic climate. (Fig. 4)
- **MLP Architecture:** Validation performance peaked with a medium-width architecture (100, 50). Increasing depth beyond three layers resulted in vanishing gradients, while narrower layers failed to capture the non-linear interactions between demographic features.

MLP Architecture Comparison. To further quantify the effect of network width and depth, we evaluated four architectures: $(32,)$, $(64,)$, $(64, 32)$, and $(128, 64)$. (Fig. 3) The resulting validation AUC–PR scores were 0.6625, 0.6690, 0.6675, and 0.6525, respectively. The single-layer 64-unit network achieved the strongest performance, while deeper or wider architectures did not yield improvements. In particular, the largest configuration (128, 64) underperformed, suggesting mild overfitting. These results indicate that, for this dataset, moderate-capacity networks generalize better than deeper or more expressive architectures.

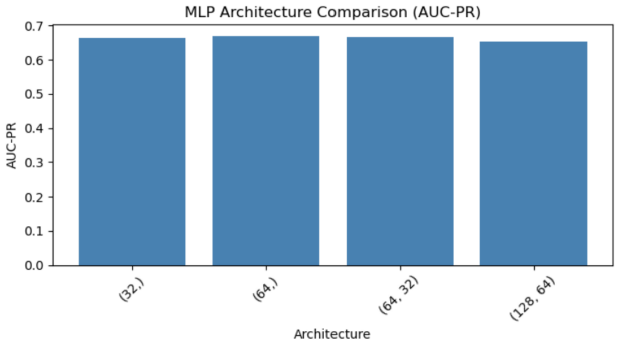

Figure 3: MLP Architecture Comparison

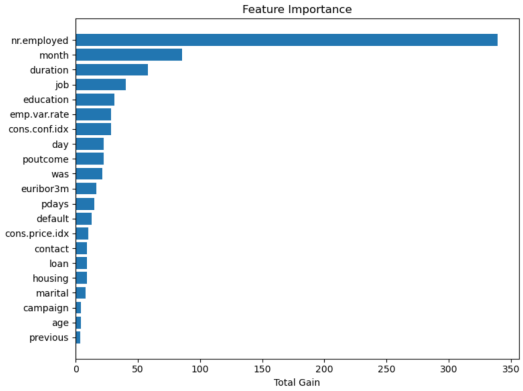

Figure 4: XGBoost Feature Importance

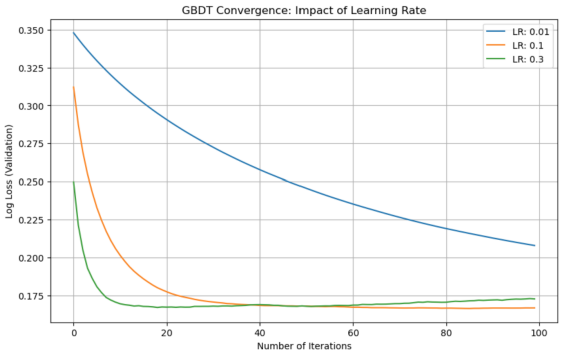

Figure 5: Impact of Learning Rate on Convergence

5 GBDT vs MLP Comparison

This section provides a direct comparison between Gradient Boosted Decision Trees (XGBoost) and Multi-Layer Perceptrons (MLP) using identical preprocessing, data splits, and evaluation metrics. We analyze predictive performance, training efficiency, interpretability, handling of feature types, and sensitivity to hyperparameters.

5.1 Performance Comparison

Table 1 summarizes the validation and test performance of XGBoost and MLP. XGBoost achieves the highest AUC-PR on both splits, indicating superior ranking performance on this imbalanced dataset. MLP performs competitively but consistently below XGBoost.

Model	Split	Acc	Prec	Rec	F1	AUC-PR
XGBoost	Val	0.917	0.744	0.397	0.517	0.710
	Test	0.913	0.698	0.402	0.510	0.667
MLP	Val	0.912	0.667	0.379	0.483	0.667
	Test	0.911	0.653	0.391	0.491	0.638

Table 1: Side-by-side comparison of XGBoost and MLP performance.

XGBoost outperforms MLP across all metrics, with the largest margin observed in AUC-PR. Both models achieve similar accuracy due to the dataset's class imbalance, but XGBoost provides higher precision and F1-score, indicating better discrimination of the minority class.

5.2 Training Time

XGBoost trains significantly faster (0.62 seconds) than MLP (6.29 seconds). This difference arises because:

- XGBoost with `tree_method=hist` is highly optimized for tabular data.
- MLP requires iterative gradient-based optimization over thousands of parameters.

For large-scale or time-sensitive applications, XGBoost offers a clear computational advantage. The order-of-magnitude difference in training speed (0.62s vs 6.29s) is attributed to XGBoost's optimized histogram-based tree construction, which is inherently more efficient for tabular data than the iterative backpropagation and weight updates required by the MLP's architecture.

5.3 Interpretability

XGBoost provides gain-based feature importance scores, enabling direct inspection of which features drive predictions. The top features include job categories, marital status, education, contact type, and economic indicators. These insights are useful for understanding customer behavior and informing marketing strategy.

In contrast, MLP behaves as a black-box model. Although techniques such as SHAP or integrated gradients could be applied, they are not as straightforward or computationally efficient as tree-based importance measures. For stakeholders who require transparent decision-making, XGBoost is more interpretable.

5.4 Handling of Categorical and Missing Data

XGBoost naturally handles missing values by learning default split directions, and it performs well with one-hot encoded categorical variables. It is robust to heterogeneous feature types and does not require scaling.

MLP, however, requires:

- imputation of missing values,
- one-hot encoding of categorical variables,
- standardization of numerical features.

This additional preprocessing increases complexity and introduces more opportunities for data leakage if not handled carefully.

5.5 Sensitivity to Hyperparameters

XGBoost shows moderate sensitivity to learning rate and tree depth. Our sweeps indicate that:

- learning rate = 0.1 provides the best balance between stability and convergence,
- depth = 6 yields the strongest performance, with depth 3 underfitting and depth 9 slightly overfitting.

MLP is more sensitive to hyperparameters, particularly:

- learning rate (0.001 performs best),
- activation function (ReLU outperforms Tanh),
- network depth and width.

Improper settings can lead to slow convergence or unstable training, making MLP more challenging to tune.

5.6 Summary

Overall, XGBoost demonstrates superior predictive performance, faster training, and greater interpretability compared to MLP on this dataset. MLP remains a viable alternative, especially when richer feature representations or larger datasets are available, but for structured tabular data with mixed feature types, GBDT models offer a strong and efficient solution.

6 Discussion

The results of this study highlight several important insights about model behavior on structured, imbalanced tabular data. Across all evaluation metrics, XGBoost consistently outperformed the MLP, with the largest margin observed in AUC-PR. This aligns with well-established findings in the machine learning literature: tree-based boosting methods tend to excel on heterogeneous tabular datasets, particularly when interactions between categorical and numerical features are important. The strong performance of XGBoost suggests that the dataset contains non-linear feature interactions and threshold-based decision patterns that are naturally captured by decision trees.

MLP performance, while competitive, lagged behind XGBoost on both validation and test sets. Neural networks typically require large amounts of data, careful regularization, and extensive hyperparameter tuning to match the performance of boosting methods on tabular tasks. The MLP's sensitivity to learning rate,

activation function, and architecture was evident in our experiments. Although the chosen configuration achieved stable training and reasonable generalization, the model did not match the representational efficiency of XGBoost. This reinforces the idea that MLPs are not always the most effective choice for tabular data unless substantial feature engineering or larger datasets are available.

The comparison also highlights differences in interpretability. XGBoost provides clear gain-based feature importance scores, enabling practitioners to understand which variables drive predictions. This is particularly valuable in domains such as finance and marketing, where transparency is often required. In contrast, the MLP behaves as a black-box model, and interpreting its learned representations would require additional tools such as SHAP or gradient-based attribution methods. For stakeholders who prioritize interpretability, XGBoost offers a more accessible explanation framework.

From a computational perspective, XGBoost trained an order of magnitude faster than the MLP. This efficiency is largely due to the optimized histogram-based tree construction algorithm and the relatively small number of boosting rounds required for convergence. The MLP, by contrast, required iterative gradient updates over thousands of parameters and multiple epochs. For real-time or large-scale applications, the faster training time of XGBoost may be a practical advantage.

Despite these strengths, both models have limitations. XGBoost can overfit when tree depth is too large or when learning rates are not properly tuned. MLPs require careful preprocessing and are sensitive to feature scaling and initialization. Additionally, neither model directly addresses the class imbalance inherent in the dataset; future work could incorporate techniques such as focal loss, class weighting, or resampling strategies to further improve minority-class recall.

Overall, the study demonstrates that XGBoost is a strong and reliable choice for this prediction task, offering superior performance, faster training, and greater interpretability. MLPs remain a flexible alternative, particularly in settings where richer feature representations or larger datasets are available, but they require more careful tuning to achieve competitive results. The findings reinforce the importance of selecting model families that align with the structure and scale of the data.

7 AI Tool Usage Disclosure

In accordance with the assignment guidelines, I disclose the use of AI tools during the completion of this project. AI assistance was used to support specific parts of the workflow, while all conceptual decisions, coding, experimentation, and interpretation were performed by me.

Tools Used

I used large language models (LLMs), including Microsoft Copilot, to assist with:

- drafting and refining explanatory text for the report,
- generating initial code templates,
- and debugging code-related issues.

My Contributions

All decisions regarding data cleaning, feature engineering, preprocessing design, model selection, hyperparameter choices, and interpretation of results were made by me. I executed the full workflow in the notebook, ran all experiments, generated all plots and tables, and developed the analytical insights and conclusions based on my understanding of the material.